Spectre: Closed-Loop Refinement, Repair, and Conformance for Rust

Anand Kumar Keshavan[0009−0007−8541−5203]

Independent Researcher, India akkeshavan@gmail.com

Abstract. We present Spectre, a formal-methods toolchain that derives a refinement mapping from Rust source and maintains it throughout model checking, counterexample-guided repair, conformance testing, and runtime monitoring. AST-based mining from a Rust `struct`/`impl` pair produces both a Spectre model and a Rust – model field/action correspondence (*refinement map*) with no manual model-skeleton or correspondence effort. A CEGIS repair engine synthesises weakest-precondition guards for simultaneous assignments, Boolean/enum conditions, and aggregates constraints across multiple counterexamples into conjunctive guards. The refinement map drives automatic generation of ITF conformance tests, Rust `#[test]` regression tests, and coverage-guided MBT traces (action, transition-pair, boundary, rare-action, and property-directed modes). `spectre sync` detects drift by comparing fields, actions, guards, and normalised assignment expressions, detecting **30/35 injected mutations** (85.7%); when one action changes, `verify --incremental` incrementally updates and re-verifies the reachable-state graph—**3.0–3.6× faster** than cold BFS on the 22,432-state Raft model (8.7 s cold BFS on our benchmark platform, 2.4–2.9 s incremental); cache restore for an unchanged spec takes 0.26 s (**33× faster**). Direct simulation (`spectre simulate`) samples execution paths without constructing the full graph, covering 5- and 7-node Raft configurations beyond BFS scale. Evaluation on 10 Rust components confirms 36/36 guards correctly mined, 10/10 positive MBT traces replayed, 2/2 injected implementation faults detected, and WP repair in under 6 s across arithmetic, Boolean, enum, and simultaneous assignment forms.

Keywords: Formal specification · Refinement · Program synthesis · CEGIS · Model-based testing · Runtime monitoring · Rust

1 Introduction

Formal specification has demonstrated its value at scale: Amazon Web Services reports using TLA+ and model checking to uncover subtle design errors in critical distributed systems before implementation and deployment [16]. Yet adoption remains narrow. The principal barrier is not expressiveness but ergonomics: TLA+ uses mathematical notation unfamiliar to systems programmers [25], and even Quint—which maps TLA+ concepts to a TypeScript-like syntax—remains specification-first: the implementation–model bridge is supplied separately by the

user, and neither Quint nor TLA+ provides deterministic weakest-precondition repair or a Rust-AST-derived correspondence.

Three practical limitations restrict wider adoption. First, *specification authoring cost*: expressing a full system model in an unfamiliar notation, starting from scratch. Second, *manual guard discovery*: when the model checker reports a violation, identifying and inserting the missing precondition by hand. Third, *specification drift*: once verified, specification and implementation can evolve independently with limited tooling to detect or recover from divergence.

We present Spectre, an open-source formal-methods toolchain that addresses all three stages. Unlike specification-first workflows (Quint, TLA+) where the model–implementation bridge is manually supplied, Spectre derives this correspondence from Rust source and reuses it throughout verification, repair, testing, synchronisation, and runtime monitoring. Spectre's central contribution is a *source-derived refinement map* that connects Rust implementation states and transitions to the formal model, and is maintained across the full development lifecycle.

C1: Source-derived refinement and incremental synchronisation. `mine -o` produces a Spectre model plus a Rust–model field/action correspondence from Rust ASTs. `sync` classifies code changes as *structurally unchanged*, *spec-update-required*, or *possibly-unsafe*, comparing fields, actions, guard counts, guard expressions, and normalised assignment expressions; semantic changes not visible in the current normalised representation remain a limitation. `drift` detects bidirectional staleness.

C2: Counterexample-guided repair with validation. WP synthesis over simultaneous assignments, Boolean/enum guards, and multiple counterexamples; trace-preserving repair validation that flags over-restrictive guards before application.

C3: Generated conformance and regression infrastructure. `spectre check-refinement` classifies implementation transitions as legal, stuttering, or violating; coverage-guided ITF trace generation (action, transition-pair, boundary, rare-action, and property-directed modes); automatic `#[test]` generation from counterexamples.

C4: Scalable exploration. `spectre simulate` for direct path sampling without full graph construction; `param N: int` for parameterised models enabling scale-sweep verification (e.g., 3-/5-/7-node Raft).

Sections 2–5 present the architecture, repair, sync, and testing components; §6 evaluates; §7–8 discuss related work and conclude.

2 Source-Derived Refinement Architecture

Kripke Structures and Bounded Checking. A *Kripke structure* $M = (S, S_0, R, L)$ has states S, initial states $S_0 \subseteq S$, a transition relation $R \subseteq S \times S$, and a labelling function L [4]. Spectre constructs M_Σ by BFS from S_0, evaluating all enabled actions and checking invariants on each transition. `always(P)` requires P in every

explored state. `eventually(P)` reports a *bounded witness*: $\exists \pi.\, \exists i \leq k$: $M, \pi_i \models P$, i.e., bounded existential reachability, not the universal LTL property $M \models \Diamond P$ [19]. Full automata-based LTL over infinite paths and WF/SF checking are future work.

The Refinement Map. Let Σ_M be the Spectre model state space and Σ_I the Rust implementation state space. A *refinement map* $\mathcal{R} \subseteq \Sigma_I \times \Sigma_M$ relates implementation states to model states via a field projection π: for $(s_I, s_M) \in \mathcal{R}$, each spec variable maps to the corresponding Rust field value. An implementation transition $(s_I \to t_I)$ *conforms* to $\mathcal{R}$ if either: (i) there exists a model action A such that $(s_I, s_M) \in \mathcal{R}$, $(t_I, t_M) \in \mathcal{R}$, all `require` guards of A hold in s_M, and $s_M \xrightarrow{A} t_M$ is a legal Spectre transition; or (ii) the step is a *stuttering step*: $\pi(s_I) = \pi(t_I)$ (all spec-relevant fields unchanged).

Proposition 1 (Refinement Conformance). *If `check-refinement` returns* Legal *for a trace transition and the field/action map is faithful, then the corresponding Spectre action is enabled in s_M, all `require` guards hold in s_M, and $\pi(t_I) = t_M$ (projected post-states agree).*

Scope. This is trace-level conformance — it verifies every supplied transition, not all possible implementation executions.

Source Derivation. `spectre mine -o spec.spec src.rs` produces two artefacts: a `.spec` file containing the formal model, and a *refinement map file* (`.driver.json`) recording the field/method correspondence. The AST miner [24] maps struct fields to `var` declarations, constructor literals to `init` values, `assert!` calls and early-return guards to `require` guards, and field assignments to primed transitions. This derivation is the key automation enabling automatic refinement checking: no user-supplied state-extraction or transition-reproduction code is required. The mined model faithfully reflects the Rust implementation; correctness properties (`invariant` and `temporal` declarations) must be supplied independently to avoid a circular oracle where implementation bugs are reproduced verbatim in the model.

Spectre Language. A Spectre specification declares typed state variables with `var`, an initial state with `init`, and guarded actions with `action`. An action lists `require` guards (preconditions) followed by primed-variable assignments (`x' = expr`). Disabled actions are silently skipped; unmentioned variables retain their value (frame condition). Parameterised models declare `param N: int` and bind values at verification time via `--param N=5`, enabling scale-sweep experiments. Actions with `int` parameters are instantiated over $[-5, 5]$ by default (`--param-bound` configurable); `--max-states` and `--max-depth` cap exploration.

Table 1 compares Spectre with TLA+ and Quint across the features most relevant to the closed-loop lifecycle. Figure 1 shows the full pipeline.

Table 1. Feature comparison of formal specification tools (open-source/core toolchains; commercial services excluded).

Feature	TLA+	Quint	Spectre
Infinite-trace temporal checking	✓	✓	–$^{\diamond}$
Bounded temporal checking	✓	✓	✓
Type inference	–	✓	✓
WP guard synthesis	–	–	✓
Source-derived Rust – model map	–	partial†	✓
Incremental sync / drift detection	–	–	✓
Runtime monitor generation	–	–	✓
Coverage-guided MBT traces	–	○*	✓
Parameterised models	✓	✓	✓
Self-loop / livelock detection	–	–	✓

†Quint Connect requires user-supplied state extraction and step mapping.
$^{\diamond}$Bounded-prefix checking; full LTL, WF, and SF are future work.
*Quint: random sim. + `--n-traces`; action/boundary/rare-action modes absent.

3 Counterexample-Guided Repair with Trace-Preservation Validation

Algorithm. Spectre implements a counterexample-guided refinement loop [22, 1] for guard synthesis. Given violated invariant *inv* and counterexample ending with action A in pre-state σ_{k-1}, Spectre computes $g = \mathrm{wp}(A, inv)$ [5] by syntactic substitution. The candidate guard is added as a `require` clause and the model is re-explored; the loop iterates until no violation is found within the configured state budget or a fixed-point is reached. The iterative loop is not guaranteed to converge in general; in practice each iteration eliminates at least the current counterexample.

Extended Synthesis. All assignment forms are handled uniformly by substitution. For an action with simultaneous assignments $\{x'_1 = e_1, \ldots, x'_n = e_n\}$ and invariant I:

$$\mathrm{wp}(\{x'_i = e_i\},\ I) \ =\ I[x_1 \leftarrow e_1,\ \ldots,\ x_n \leftarrow e_n]$$

Boolean ($b' = \mathit{true}/\mathit{false}$) and enum ($e' = V$) assignments are syntactic instances of $x' = e$: after substitution the expression may reduce to a constant, in which case the guard is omitted (*true*) or the action is statically disabled (*false*). `AccumulateGuards` aggregates distinct WP conditions across counterexamples for the same (invariant, action) pair.

Formal Guarantees.

Proposition 2 (Substitution Soundness). *Let I be an invariant and A an action with simultaneous assignments $\{x'_1 = e_1, \ldots, x'_n = e_n\}$. Let $g = I[x_1 \leftarrow e_1, \ldots, x_n \leftarrow e_n]$ be the result of simultaneous syntactic substitution. For any*

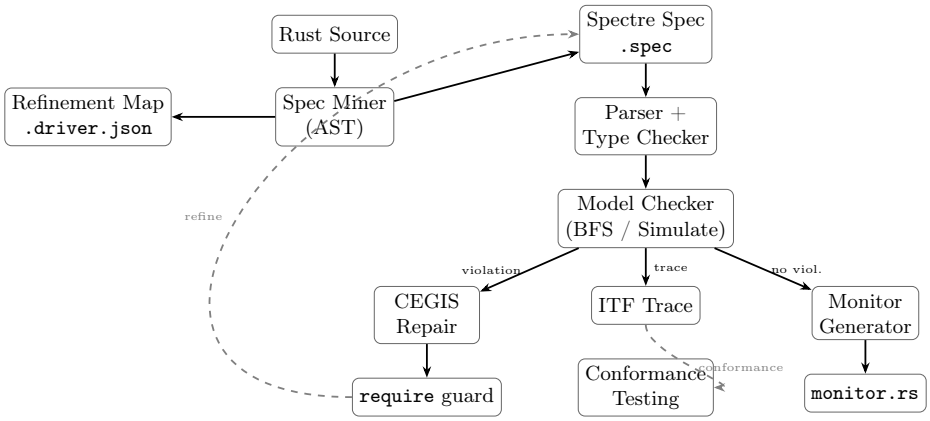

Fig. 1. Spectre architecture. `spectre mine` produces both the Spectre model and the Refinement Map (`.driver.json`) from the Rust AST. The model checker feeds violations to CEGIS repair and traces to conformance testing; `sync` and `drift` use the Refinement Map for incremental re-verification.

pre-state σ: if $\sigma \models g$ then $A(\sigma) \models I$. This holds for all supported assignment forms; arithmetic, Boolean, and enum assignments are syntactic instances.

Corollary 1 (Trace-Preserving Repair Validation). *Let g be a guard synthesised by WP substitution for assignments $\{x_i' = e_i\}$ against invariant I. (i)* ***Soundness.*** *Adding g as a* `require` *clause prevents the violating transition. (ii)* ***Trace preservation.*** *The* `--check-repair-minimal` *checker validates that g does not block any transition present in supplied positive traces; a guard that would block a known-good step is flagged as over-restrictive. This is positive-trace preservation, not global minimality.*

Causal Diagnosis. For each counterexample, Spectre computes: the first divergence point (earliest step where the WP condition was unsatisfied), the responsible variable set (variables referenced in the violated invariant clause that changed along the path), and the candidate fix in Spectre syntax. This structured output replaces verbose trace dumps and is reported immediately after each violation.

4 Incremental Synchronisation and Verification

Code and specifications drift. Spectre provides three mechanisms to detect and recover from drift without full re-exploration.

Bidirectional Drift Detection (`spectre drift`). The refinement map file records field and method names at mine time. `spectre drift` compares the current Rust source against this record and reports: (a) Rust fields with no matching spec variable; (b) spec variables with no Rust implementation correspondence;

(c) invariants that may need monitor regeneration. The check is $O(|F| + |A|)$ in the number of fields and actions and completes in milliseconds.

Incremental Synchronisation (`spectre sync`). `spectre sync` re-mines the Rust source and classifies each change against the existing spec:

- *Structurally unchanged*: same variables, same actions, same guard counts, same guard expressions, and canonicalised-AST-string-equal assignment bodies (printed AST after constant folding) — no specification update required.
- *Spec-update-required*: new or renamed variables/actions — affected spec sections must be updated before re-verification.
- *Possibly-unsafe*: changed guard or assignment expressions (wrong arithmetic operator, wrong comparison, or omitted guard), or removed variables/actions — immediate re-verification recommended.

The command exits with status 1 if any non-preserving changes are found, making it suitable as a CI gate.

Cached and Incremental Re-verification. Spectre hashes the spec content (SHA-256) and persists the BFS graph under `.spectre-cache/`. On the next run, a matching hash avoids re-exploration ($\sim$33$\times$ faster for Raft: 0.26 s vs. 8.7 s on our benchmark platform). When `sync` flags one action as *possibly-unsafe*, `verify --incremental` re-verifies only that action: stale transitions are pruned, reachability is recomputed from initial states, now-unreachable states are removed, the action is re-executed from all remaining reachable states, and newly reached states are BFS-expanded up to the original state bound. Table 6 reports results; speedup varies with the fraction of states reachable only via the changed action.

Proposition 3 (Cache Soundness). *Let* $H = \text{SHA-256}(\mathit{spec} \parallel \mathit{params} \parallel \mathit{limits} \parallel \mathit{version})$. *If* H *matches the cached fingerprint, the graph faithfully represents reachable states under those parameters, bounds, and interpreter semantics.*

Proposition 4 (Incremental Equivalence). *Let* $\mathcal{C}$ *be an exploration configuration (depth limit, state limit, parameter bounds, and deterministic BFS action order), and let* $G_{\mathcal{C}}(S)$ *denote the graph produced by* Spectre*'s BFS of spec* S *under* $\mathcal{C}$*. Let* S' *differ from* S *only in action* A*'s transition relation. After running* IncrementalReverify *on* $(G_{\mathcal{C}}(S), A, S', \mathcal{C})$*:* (i) *all* A*-transitions are replaced by those of* S'*;* (ii) *states reachable only via* A *in* S *are removed;* (iii) *new states reachable via* A *in* S' *are BFS-expanded under all actions within* $\mathcal{C}$*. The resulting graph equals* $G_{\mathcal{C}}(S')$*, assuming all actions other than* A *are unchanged and the BFS interpreter is deterministic.*

5 Generated Conformance Testing

The source-derived refinement map drives three complementary testing outputs, all automatically derived from the same pipeline artefacts.

Refinement Checking. The `check-refinement` subcommand (with `--traces`) classifies each transition in each ITF trace [11] as: *Legal step* (action found in the refinement map, all `require` guards satisfied in the pre-state), *Stutter step* (all spec-relevant fields unchanged), or *Violation* (neither). Violations include a structured diagnosis: which guard failed, the pre-state values, and the responsible action.

Coverage-Guided ITF Traces. The `verify --emit-traces --coverage-mode` flag generates ITF traces optimised for five modes: *action coverage* (default: prefer unseen actions); *transition-pair* (prefer unseen (prev, curr) action pairs); *boundary* (steer toward integer variable min/max); *rare-action* (prefer least-frequently-taken transitions); *property* (steer toward states where the minimum integer variable is lowest, i.e., nearest to a lower-bound invariant boundary; the current prototype supports integer lower-bound invariants of the form $x \geq c$, preferring states minimising $x - c$).

Table 2 compares coverage modes on the `bank-account-corrected` spec (20-transition budget; invariant: `aliceBalance` ≥ 0). Property mode achieves the lowest mean `aliceBalance` (23.8) and spends the most steps at the invariant boundary (`aliceBalance`$= 0$). Boundary mode underperforms because it targets min/max equally and reaches very high balance values; action mode produces a shorter trace once all actions are covered.

Table 2. Coverage-mode comparison on `bank-account-corrected` (20-transition budget). "Trace steps" counts the initial state plus transitions. "Boundary steps" = steps with `aliceBalance`$= 0$. *action* mode stops early once all actions are covered.

Mode	Trace steps	Mean `aliceBalance`	Boundary steps
action	8	41.2	2/8 (25%)
boundary	21	500.0	1/21 (5%)
rare-action	21	30.5	5/21 (24%)
property	**21**	**23.8**	**11/21 (52%)**

Near-Zero-Driver MBT and Regression Tests. When `spectre mine -o` writes the refinement map file, `spectre generate-driver` emits a fully-connected Rust `StepHandler` that dispatches directly to the real Rust methods; the only remaining manual step is one `use` import per component. `spectre verify –emit-traces` generates positive witness traces that exercise each action at least once; replaying these confirms the implementation accepts all model-legal sequences.

6 Evaluation

We evaluate Spectre around five research questions. Hardware: Apple M3 Pro (`aarch64`), 18 GB RAM, macOS 15.6, Go 1.24 (`go install`, default op-

Table 3. Spec mining and refinement map derivation across 10 Rust components. "Guards" = `require` clauses in the mined spec (manually verified correct); "Correct?" = guards semantically correct vs. source intent.

Component	Fields	Actions	Asserts	Guards	Correct?
`BankAccount`	3	4	7	7	✓
`Counter`	3	3	2	2	✓
`Stack`	3	3	2	2	✓
`Queue`	4	3	2	2	✓
`RateLimiter`	3	3	3	3	✓
`CircuitBreaker`	4	4	5	5	✓
`ConnectionPool`	3	4	4	4	✓
`Semaphore`	3	4	4	4	✓
`EventLog`	4	5	6	6	✓
`Cache`	4	4	1	1	✓
Total	34	37	36	36	10/10

timization); default limits 5,000 states/depth 100. All wall-clock times are on this platform; reported ratios compare same-platform runs. Reproduction: `HOW-TO-RUN-BENCHMARKS.md` (Docker or local build).

RQ1: How accurately does Spectre derive the Rust – model correspondence? We evaluated mining on 10 Rust components: bank account, counter, stack, queue, rate limiter, circuit breaker, connection pool, semaphore, event log, and cache (sources in `examples/rust/`). Table 3 reports per-component field, action, and guard counts.
Guards = Asserts in every row confirms **100% recall**: all 36 `assert!` guards are detected and semantically correct (0 false guards), and every mined spec runs under `spectre verify` without modification. Each component also produced a correct refinement map file (10/10), enabling automatic driver generation, conformance checking, and regression-test generation. The corpus is limited to author-controlled components; evaluation on independently developed crates is future work.

RQ2: How effectively does repair synthesise safe, trace-preserving guards? Table 4 reports WP repair on five benchmark specs spanning arithmetic, Boolean, enum, and simultaneous assignment forms. All five cases are handled by the same substitution rule. Symbolic WP computation takes under 1 ms; the end-to-end repair time (synthesis plus BFS re-exploration per CEGIS iteration) ranges from $<$0.01 s to 5.5 s, with the BankAccount outlier incurring three 5,000-state BFS passes.
Boolean and enum cases simplify after substitution (e.g. $\mathrm{wp}(e' = V,\ !(e = V \wedge c = 0)) = !(c = 0)$); each is sound by Proposition 2.

RQ3: How effectively do generated tests detect implementation divergence? Table 5 reports results for the near-zero-driver MBT evaluation. All 10 drivers

Table 4. WP repair benchmark across assignment forms. "CEs" = CEGIS iterations; "E2E time" = end-to-end repair time (synthesis + re-exploration).

Spec	Form	CEs	E2E time (s)	Guard (abbreviated)
`bank-account-violation`	arith	3	5.5	`bal - 30 >= 0`
`inventory-violation`	arith	2	<0.01	`inv - 30 >= 0`
`repair-bool`	bool	1	<0.01	`!(count = 0)`
`repair-enum`	enum	1	<0.01	`!(result = 0)`
`repair-simultaneous`	simul	1	<0.01	`bal - 30 >= 0`

were generated without manual stubs; the only user-written line is one `use` import per component. Positive trace replay passed for all 10 components; `CircuitBreaker`'s enum state required a one-time serialization mapping (4 lines), which is the only non-generated code across the suite. Both injected faults were detected: the balance-doubling bug produced 3 field mismatches across 4 trace steps; the decrement-direction bug produced 1 mismatch at step 2. The same counterexamples also produced compilable `#[test]` regression functions via `--emit-rust-tests`.

To evaluate `spectre sync` mutation detection, we injected 35 mutations across the 10 components (types: guard-omit, wrong-arith, wrong-cmp, bool-invert). `spectre sync` detected **30/35** (85.7%) mutations: all 16 guard-omit mutations (guard count change), all 10 wrong-arith mutations (assignment body change: e.g. `count+step` → `count-step`), 3/5 wrong-cmp mutations (guard expression string change), and 1/4 bool-invert mutations—leaving 5 undetected (2 wrong-cmp and 3 bool-invert). Canonicalised AST-string equality after constant folding covers the wrong-arith cases that structural comparison previously missed; the five remaining mutations produce semantic changes not represented distinctly by the current normalised comparison (guard-level rewrites and polarity inversions not reducible to assignment-body string differences).

RQ4: How much does caching and incremental re-verification reduce cost? Table 7 reports exploration results. For `raft-election-safety` (22,432 states, 8.7 s cold BFS), cache restore on an unchanged spec takes 0.26 s—a ∼33× reduction; for bank-account (3,745 states, 2.1 s cold BFS), 0.18 s (11×). Table 6 details incremental re-verification results when one action changes. Speedup ranges from 3.0× (Raft `vote2for1`, which must re-explore ≈6,493 newly reachable states) to 7.7× (bank-account `freeze`, where all pruned states are immediately recovered by re-executing freeze with no further BFS expansion needed). The scientific result is the ratio and the work avoided; absolute seconds will shift on other hardware. Cache soundness follows from Proposition 3; incremental equivalence follows from Proposition 4. To validate empirically, we ran fresh full BFS after each incremental re-verification and confirmed identical reachable-state and transition-set hashes in all benchmark cases (bank-account and Raft).

Table 5. Near-zero-driver MBT and generated test evaluation. "Import LOC" = `use` imports (1 per component); "Adapter LOC" = non-generated serialization code (0 except CircuitBreaker enum); "Positive trace" = ITF replay success; "Injected fault" = fault detected.

Component	Driver gen?	Import LOC	Adapter LOC	Positive trace	Injected fault
`BankAccount`	✓	1	0	✓	detected
`Counter`	✓	1	0	✓	detected
`Stack`	✓	1	0	✓	–
`Queue`	✓	1	0	✓	–
`RateLimiter`	✓	1	0	✓	–
`CircuitBreaker`	✓	1	4	✓	–
`ConnectionPool`	✓	1	0	✓	–
`Semaphore`	✓	1	0	✓	–
`EventLog`	✓	1	0	✓	–
`Cache`	✓	1	0	✓	–
Total	10/10	10	4	10/10	2/2

Table 6. Incremental re-verification vs. cold BFS (same platform). *Pruned* = states removed as unreachable without the changed action (deterministic); *new BFS* = newly BFS-expanded states (varies ±2 across runs due to hash-map iteration order). Total states always equals the original count (verified by hash).

Spec	Action	Cold	Incr.	Speedup	Pruned / new BFS
bank-account (3,745)	`freeze`	2.1 s	0.27 s	7.7×	1,882 / 0
Raft (22,432)	`stepDown1_s2`	8.7 s	2.4 s	3.6×	1,649 / ≈1,644
Raft (22,432)	`vote2for1`	8.7 s	2.9 s	3.0×	6,501 / ≈6,493

Case Study: Raft Election Safety. We applied Spectre to a Rust Raft consensus node [17] (`examples/rust/raft.rs`; 8 methods, 9 `assert!` guards). The AST miner extracted the per-node skeleton in a single pass; we composed three instances into a 3-node cluster spec. Exhaustive BFS of this bounded abstraction explored **22,432 reachable states** (8.7 s on our benchmark platform) and found no violations of *electionSafety* (at most one leader per term), *leaderMajority* (≥ 2 of 3 votes), or *candidateSelfVote*. Terms and log lengths are bounded 0–3; this is a finite abstraction of the election sub-protocol, not a full Raft verification. `spectre simulate` can sample beyond this bound when parameterised models push BFS to its state limit.

RQ5: How does Spectre scale from exhaustive to simulation? `spectre simulate -traces N` samples N paths on demand without a pre-built graph, with invariant checking on each step. Table 8 reports results on 3-/5-/7-node Raft. On our benchmark platform, the 3-node spec is fully explored (22,432 states, 8.7 s). The 5-node spec exceeds the BFS limit (50,000 states, 4m 29s); simulation finds no

Table 7. Verification results. †State limit hit; result is *No CE found (bounded)*, not exhaustive. ‡Capped at 100; unbounded resource var makes BFS impractical. §3-node cluster; per-node skeleton mined from Rust.

Specification	States	Time (s)	Result
`bank-account-violation`	5,000†	1.26	Counterexample found
`bank-account-corrected`	5,000†	0.33	No CE found (bounded)
`concurrent-lock-violation`	100‡	0.26	Counterexample found
`concurrent-lock-corrected`	1,928	0.18	Verified (exhaustive)
`counter`	100†	0.01	Counterexample found
`message-queue-corrected`	22	<0.01	Verified (exhaustive)
`inventory-corrected`	11	<0.01	Verified (exhaustive)
`stuttering-counter`	100†	0.01	Counterexample found
`raft-election-safety`§	22,432	8.7	Verified (exhaustive)

violations in 7.5 s across 1,000 paths. The 7-node spec is not run under BFS; simulation covers 1,000 paths in 27.6 s.

Table 8. 3-/5-/7-node Raft scalability. BFS limit = 50,000 states. Simulation = `spectre simulate -traces 1000`.

Config	Variables	Actions	BFS result	Sim time	Violations
3-node	15	27	22,432 states, 8.7 s	–	–
5-node	25	75	>50k, 4m29s	7.5 s	0/1000
7-node	35	147	infeasible	27.6 s	0/1000

Runtime Monitor. `generate-monitor` produces a self-contained `monitor.rs` with no external dependencies; `Monitor::step` performs $O(k)$ work with no heap allocation. Benchmarked on Apple M3 Pro (`aarch64`, Rust 1.88, `-C opt-level=3`, best of 5×10^7 calls), cost ranged from 3.7 ns ($k = 1$) to 6.1 ns ($k = 20$); all checks are stack-resident.

7 Related Work

Specification-First vs. Implementation-First. TLA+/TLC [13, 12] and Quint [20] are specification-first workflows in which the specification is authored or generated independently of an automatically derived Rust–model correspondence. Quint Connect [21] requires a separate `State::from_driver` mapping for Rust MBT; tla_connect [23] provides TLA+/Apalache ITF [11] replay for Rust via a similar separate bridge. For components with primitive and directly serialisable enum state, Spectre automatically derives this correspondence from the Rust AST,

removing the need for a separately provided bridge; more complex serialisation (e.g. CircuitBreaker's enum mapping, 4 lines) falls outside this scope. Cirstea et al. [3] validate distributed-program traces against TLA+ specifications; Maliuginas and Petrauskas [15] translate Elixir/BEAM programs toward TLA+ specifications. Spectre pursues the complementary *implementation-first* workflow: it derives both the model and the implementation–model correspondence from Rust source, then reuses that correspondence for verification, repair, testing, synchronisation, and monitoring.

Verification Languages and Guard Synthesis. CEGIS [22]/SyGuS [1] use grammar-guided synthesis; Spectre computes WP candidates by syntactic substitution—no grammar, no search. Houdini [7] weakens candidates iteratively; Daikon [6] detects invariants from traces. Ivy [18] generalises counterexamples via SMT for infinite-state models; Spectre targets bounded finite models with concrete `require` guards. SPIN [8] operates on Promela models; related Modex work [9] extracts verification models from C source, whereas Spectre targets Rust and additionally derives a persistent correspondence used for repair, testing, synchronisation, and monitoring. Dafny [14] neither mines from Rust source nor generates runtime monitors. MOP [2]/JavaMOP [10] use separate monitor DSLs; Spectre derives monitors from the same spec used for verification.

8 Conclusion

Spectre demonstrates that a source-derived refinement map can serve as persistent verification infrastructure across the Rust development lifecycle. Mining from ten Rust components produced 36/36 correct guards, 10/10 positive MBT trace replays, and 30/35 sync mutations detected (85.7%); the only non-generated code across the suite is a 4-line enum serialization mapping for CircuitBreaker. Both injected implementation faults were detected by refinement replay. BFS verified a 22,432-state Raft abstraction in 8.7 s on our benchmark platform; cache restore on an unchanged spec takes 0.26 s (33× faster); incremental re-verification of a single changed action takes 2.4–2.9 s (3.0–3.6× faster), with the speedup depending on the fraction of states reachable only via the changed action. WP repair synthesised sound guards in under 6 s across arithmetic, Boolean, enum, and simultaneous assignment forms. Counterexample-guided repair, refinement checking, coverage-guided conformance tests, incremental synchronisation, and simulation provide complementary mechanisms for detecting and maintaining refinement correspondence as code and specifications evolve. The tool, artifact, and Docker image are available at `https://doi.org/10.5281/zenodo.21871625` (version DOI; SHA-256 in the artifact submission form). Source and examples are on GitHub at `https://github.com/akkeshavan/spectre`.

References

1. Alur, R., Bodik, R., Juniwal, G., Martin, M.M.K., Raghothaman, M., Seshia, S.A., Singh, R., Solar-Lezama, A., Torlak, E., Udupa, A.: Syntax-guided synthesis. In: Formal Methods in Computer-Aided Design (FMCAD). pp. 1–17 (2013)
2. Chen, F., Roşu, G.: MOP: An efficient and generic runtime verification framework. In: ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA). pp. 569–588 (2007)
3. Cîrstea, H., Kuppe, M.A., Loillier, T., Merz, S.: Validating traces of distributed programs against TLA+ specifications. In: Software Engineering and Formal Methods (SEFM). LNCS, vol. 15280, pp. 126–143. Springer (2024). https://doi.org/10.1007/978-3-031-77382-2_8
4. Clarke, E.M., Emerson, E.A., Sistla, A.P.: Automatic verification of finite-state concurrent systems using temporal logic specifications. ACM Transactions on Programming Languages and Systems **8**(2), 244–263 (1986)
5. Dijkstra, E.W.: A Discipline of Programming. Prentice-Hall (1976)
6. Ernst, M.D., Perkins, J.H., Guo, P.J., McCamant, S., Pacheco, C., Tschantz, M.S., Xiao, C.: The daikon system for dynamic detection of likely invariants. Science of Computer Programming **69**, 35–45 (2007)
7. Flanagan, C., Leino, K.R.M.: Houdini, an annotation assistant for ESC/Java. In: Formal Methods Europe (FME). LNCS, vol. 2021, pp. 500–517. Springer (2001)
8. Holzmann, G.J.: The SPIN Model Checker: Primer and Reference Manual. Addison-Wesley (2004)
9. Holzmann, G.J., Smith, M.H.: Software model checking: Extracting verification models from source code. Software Testing, Verification and Reliability **11**(2), 65–79 (2001). https://doi.org/10.1002/stvr.228
10. Jin, D., Meredith, P.O.M., Lee, C., Roşu, G.: JavaMOP: Efficient parametric runtime monitoring framework. In: International Conference on Software Engineering (ICSE). pp. 1427–1430 (2012). https://doi.org/10.1109/ICSE.2012.6227231
11. Konnov, I.: ADR-015: Informal trace format in JSON. Apalache Documentation, `https://apalache-mc.org/docs/adr/015adr-trace.html`, last revised 2025-11-11; accessed 2026-08-09
12. Kuppe, M.A., Lamport, L., Ricketts, D.: The TLA+ toolbox. In: Proceedings of the 3rd Workshop on Formal Integrated Development Environment (F-IDE). EPTCS, vol. 310, pp. 50–62 (2019)
13. Lamport, L.: Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley (2002)
14. Leino, K.R.M.: Dafny: An automatic program verifier for functional correctness. In: Proceedings of Logic for Programming, Artificial Intelligence, and Reasoning (LPAR). LNCS, vol. 6355, pp. 348–370. Springer (2010)
15. Maliuginas, A., Petrauskas, K.: Extracting TLA+ specifications out of a program for a BEAM virtual machine. Vilnius University Open Series pp. 106–114 (2024). https://doi.org/10.15388/LMITT.2024.14
16. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M.: How Amazon Web Services uses formal methods. Communications of the ACM **58**(4), 66–73 (2015)
17. Ongaro, D., Ousterhout, J.: In search of an understandable consensus algorithm. In: 2014 USENIX Annual Technical Conference (USENIX ATC). pp. 305–319 (2014)
18. Padon, O., McMillan, K.L., Panda, A., Sagiv, M., Shoham, S.: Ivy: Safety verification by interactive generalization. In: ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). pp. 614–630 (2016)

19. Pnueli, A.: The temporal logic of programs. In: 18th Annual Symposium on Foundations of Computer Science (SFCS). pp. 46–57 (1977)
20. Quint Contributors: Quint: A modern specification language for TLA+. `https://github.com/informalsystems/quint`, accessed 2026-08-09
21. Quint Contributors: quint_connect: Model-based testing bridge for Quint specifications. `https://docs.rs/quint-connect/latest/quint_connect/` (2025), accessed 2026-08-09
22. Solar-Lezama, A.: Program Synthesis by Sketching. Ph.D. thesis, University of California, Berkeley (2008), technical Report UCB/EECS-2008-176
23. tla-connect Contributors: tla_connect: TLA+/apalache ITF trace replay for Rust. `https://docs.rs/tla-connect/0.0.4/tla_connect/`, accessed 2026-08-09
24. Tolnay, D.: syn: Parser for Rust source code. `https://docs.rs/crate/syn/2.0.119` (2026), version 2.0.119. Accessed 2026-08-09
25. Wing, J.M.: A specifier's introduction to formal methods. Computer **23**(9), 8–24 (1990)